



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 大作业: 图书管理系统

专 业 班 级 通信工程 三班

学 生 学 号 _____

学 生 姓 名 _____

实 验 时 间 2025 年 5 月 25 日

实验报告

【实验目的】

实践我们这个学期所学的数据结构与算法，实现一个常见项目。

【实验原理】

对于图书管理系统，这本质上是一个数据库，对其进行最频繁的操作便是查找。无论是添加，删除，搜索，借阅等操作，都脱离不了搜索这一操作，因此，对于搜索操作，我觉得应当使用一个时间复杂度极低的算法。综合利弊，我们选择了哈希算法（散列查找），以空间换取 $O(1)$ 的时间效率。

对于我们要实现的查看全部书籍与排序算法，我们又需要线性表来打印现在图书馆全部书籍，因此我选择了数组来存储，遍历数组来实现打印全部书籍功能，最后使用快速排序来实现排序功能。（为实现多种排序方式，我们这里应该使用 C 语言自带的一些字符串函数）。

综上：我的项目是一个基于哈希表实现的图书管理系统，核心原理是通过哈希表实现高效的图书检索，同时结合数组存储图书详细信息。

【实验内容与要求】

由实验原理，结合图书管理系统的常用操作，

我完成的功能为：

功能	实现方法
图书添加	1. 哈希表检查 ISBN 是否存在 2. 存在则增加数量，不存在则创建新条目 3. 使用逻辑删除位置优化空间
图书删除	1. 标记删除而非物理删除（惰性删除） 2. 数量>1 时减少数量 3. 数量=1 时设置删除标志并从哈希表移除
图书搜索	1. 精确搜索：通过哈希表直接定位 2. 模糊搜索：线性遍历数组匹配关键词
图书排序	1. 提取有效图书到临时数组 2. 使用 qsort 按指定字段排序 3. 重建哈希表（因索引变化）
借阅管理	通过 borrowed/quantity 字段跟踪库存状态，确保 $\text{borrowed} \leq \text{quantity}$

【实验源程序】

1.Hash 头文件

```

#ifndef _Hash_h_
#define _Hash_h_

#define INITIAL_CAPACITY 7
#define LOAD_FACTOR 0.7

int isPrime(int n); //素数判断
int nextPrime(int n); //下一个素数
int hash(char* isbn, int capacity); //hash 函数
struct HashTable* createHashTable(int initialCapacity); //创建哈希表
void resize(struct HashTable* table); //调整哈希表大小
void hashInsert(char* isbn, int index); //哈希插入
int hashSearch(char* isbn); //哈希查找
void hashDelete(char* isbn); //哈希删除

#endif

```

2.Hash 函数文件

```
#include<stdio.h>
#include<stdlib.h>
#include <string.h>
#include "Hash.H"

struct HashTable* hashTable;

struct HashEntry {
    char isbn[20];
    int index;
    struct HashEntry* next;
};

struct HashTable {
    int capacity;
    int size;
    struct HashEntry** entries;
};

int isPrime(int n) {
    if (n <= 1) return 0;
    if (n <= 3) return 1;
    if (n % 2 == 0 || n % 3 == 0) return 0;
    for (int i = 5; i * i <= n; i += 6)
        if (n % i == 0 || n % (i + 2) == 0) return 0;
    return 1;
}

int nextPrime(int n) {
    if (n <= 1) return 2;
    while (!isPrime(++n));
    return n;
}

/* 哈希函数 */
int hash(char* isbn, int capacity) {
    unsigned long hash_val = 0;
    while (*isbn) {
        hash_val = hash_val * 31 + *isbn++;
    }
}
```

```

    }
    return hash_val % capacity;
}

/* 创建哈希表 */
struct HashTable* createHashTable(int initialCapacity) {
    struct HashTable* table = (struct HashTable*)malloc(sizeof(struct HashTable));
    table->capacity = isPrime(initialCapacity) ? initialCapacity :
nextPrime(initialCapacity);
    table->size = 0;
    table->entries = (struct HashEntry**)calloc(table->capacity, sizeof(struct
HashEntry*));
    return table;
}

/* 扩容重组哈希表 */
void resize(struct HashTable* table) {
    int newCapacity = nextPrime(table->capacity * 2);
    struct HashEntry** newEntries = (struct HashEntry**)calloc(newCapacity,
sizeof(struct HashEntry*));

    // 重新哈希所有条目
    for (int i = 0; i < table->capacity; i++) {
        struct HashEntry* entry = table->entries[i];
        while (entry != NULL) {
            struct HashEntry* next = entry->next;
            int newIndex = hash(entry->isbn, newCapacity);
            entry->next = newEntries[newIndex];
            newEntries[newIndex] = entry;
            entry = next;
        }
    }

    free(table->entries);
    table->entries = newEntries;
    table->capacity = newCapacity;
}

/* 哈希表插入 */
void hashInsert(char* isbn, int index) {
    // 检查负载因子
    if ((float)hashTable->size / hashTable->capacity > LOAD_FACTOR) {
        resize(hashTable);
    }
}

```

```

    int hashVal = hash(isbn, hashTable->capacity);
    struct HashEntry* newEntry = (struct HashEntry*)malloc(sizeof(struct
HashEntry));
    strcpy(newEntry->isbn, isbn);
    newEntry->index = index;
    newEntry->next = hashTable->entries[hashVal];
    hashTable->entries[hashVal] = newEntry;
    hashTable->size++;
}

/* 哈希表查找 */
int hashSearch(char* isbn) {
    int hashVal = hash(isbn, hashTable->capacity);
    struct HashEntry* entry = hashTable->entries[hashVal];

    while (entry != NULL) {
        if (strcmp(entry->isbn, isbn) == 0) {
            return entry->index;
        }
        entry = entry->next;
    }
    return -1;
}

/* 哈希表删除 */
void hashDelete(char* isbn) {
    int hashVal = hash(isbn, hashTable->capacity);
    struct HashEntry** prev = &hashTable->entries[hashVal];
    struct HashEntry* current = hashTable->entries[hashVal];

    while (current != NULL) {
        if (strcmp(current->isbn, isbn) == 0) {
            *prev = current->next;
            free(current);
            hashTable->size--;
            return;
        }
        prev = &(current->next);
        current = current->next;
    }
}

```

3.Library 函数及主程序应用示例

```
#include<stdio.h>
#include<stdlib.h>
#include <string.h>
#include "Hash.h"

#define MAX_BOOKS 1000

struct Book {
    char title[100];
    char author[50];
    char isbn[20];
    int borrowed;
    int deleted;
    int quantity;
};

struct HashEntry {
    char isbn[20];
    int index;
    struct HashEntry* next;
};

struct HashTable {
    int capacity;
    int size;
    struct HashEntry** entries;
};

// 全局变量
extern struct HashTable* hashTable;
struct Book library[MAX_BOOKS];
int bookCount = 0;

/*增加书籍*/
void addBook() {
    char isbn[20];
    printf("\n 请输入 ISBN: ");
    fgets(isbn, sizeof(isbn), stdin);
    isbn[strcspn(isbn, "\n")] = '\0';

    int index = hashSearch(isbn);
    if (index != -1 && !library[index].deleted) {
```

```

        library[index].quantity++;
        printf("已存在该 ISBN 的书籍，数量增加为%d! \n", library[index].quantity);
        system("pause");
        return;
    }

    int availableIndex = -1;
    for (int i = 0; i < MAX_BOOKS; i++) {
        if (i >= bookCount || library[i].deleted) {
            availableIndex = i;
            break;
        }
    }

    if (availableIndex == -1) {
        printf("图书馆已满，无法添加新书! \n");
        system("pause");
        return;
    }

    if (availableIndex < bookCount && library[availableIndex].deleted) {
        hashDelete(library[availableIndex].isbn);
    }

    struct Book newBook;
    printf("请输入书名: ");
    fgets(newBook.title, sizeof(newBook.title), stdin);
    newBook.title[strcspn(newBook.title, "\n")] = '\0';

    printf("请输入作者: ");
    fgets(newBook.author, sizeof(newBook.author), stdin);
    newBook.author[strcspn(newBook.author, "\n")] = '\0';

    strcpy(newBook.isbn, isbn);
    newBook.borrowed = 0;
    newBook.deleted = 0;
    newBook.quantity = 1;

    library[availableIndex] = newBook;
    hashInsert(isbn, availableIndex);

    if (availableIndex >= bookCount) {
        bookCount++;
    }
}

```

```

    printf("图书添加成功! \n");
    system("pause");
}
/*查看当前藏书*/
void displayBooks() {
    system("cls");
    printf("\n%-30s%-20s%-15s%-8s%-8s%-8s\n", "书名", "作者", "ISBN", "总数", "
在库", "借出");
    printf("-----\n");

    int found = 0;
    for(int i = 0; i < bookCount; i++) {
        if (library[i].deleted) continue;
        found = 1;
        int available = library[i].quantity - library[i].borrowed;
        printf("%-30s%-20s%-15s%-8d%-8d%-8d\n",
            library[i].title,
            library[i].author,
            library[i].isbn,
            library[i].quantity,
            available,
            library[i].borrowed);
    }

    if (!found) printf("\n 当前图书馆没有藏书! \n");
    system("pause");
}

/*查找功能*/
void searchBook_ISBN(void) {
    char isbn[20];
    int c;
    while ((c = getchar()) != '\n' && c != EOF); // 清空输入缓冲区
    printf("\n 请输入要搜索书籍的 ISBN: ");
    fgets(isbn, sizeof(isbn), stdin);
    isbn[strcspn(isbn, "\n")] = '\0';

    printf("\n 搜索结果: \n");

    int index = hashSearch(isbn);
    if (index == -1 || library[index].deleted) {
        printf("未找到该 ISBN 的图书! \n");
    }
}

```

```

        system("pause");
        return;
    }

    printf("-----\n");
    printf("书名: %s\n 作者: %s\nISBN: %s\n 总数量: %d\n 在庫: %d\n 已借出: %d\n",
        library[index].title,
        library[index].author,
        library[index].isbn,
        library[index].quantity,
        library[index].quantity - library[index].borrowed,
        library[index].borrowed);

    system("pause");
    return;
}

void searchBook_Blurred(void) {
    char keyword[100];
    printf("\n 请输入搜索关键词: ");
    int c;
    while ((c = getchar()) != '\n' && c != EOF); // 清空输入缓冲区
    fgets(keyword, sizeof(keyword), stdin);
    keyword[strcspn(keyword, "\n")] = '\0';

    printf("\n 搜索结果: \n");
    int found = 0;
    for(int i = 0; i < bookCount; i++) {
        if (library[i].deleted) continue;
        if (strstr(library[i].title, keyword) ||
            strstr(library[i].author, keyword) ||
            strstr(library[i].isbn, keyword)) {
            printf("-----\n");
            printf("书名:%s\n 作者:%s\nISBN:%s\n 总数量:%d\n 在庫:%d\n 已借出:%d\n",
                library[i].title,
                library[i].author,
                library[i].isbn,
                library[i].quantity,
                library[i].quantity - library[i].borrowed,
                library[i].borrowed);
            found = 1;
        }
    }
    if(!found) printf("未找到相关书籍! \n");
}

```

```

        system("pause");
    }

void searchBook_S(void){
    system("cls");
    int k;
    printf("查找方式: \n1.ISBN 快速查找\t2.模糊查找\n");
    printf("请输入查找方式: ");
    scanf("\n%d",&k);
    if( k == 1)
        searchBook_ISBN();
    if( k == 2)
        searchBook_Blurred();
    return;
}

/*删除功能*/
void deleteBook(void) {
    char isbn[20];
    printf("\n 请输入要删除的 ISBN: ");
    fgets(isbn, sizeof(isbn), stdin);
    isbn[strcspn(isbn, "\n")] = '\0';

    int index = hashSearch(isbn);
    if (index == -1 || library[index].deleted) {
        printf("未找到该 ISBN 的图书! \n");
        system("pause");
        return;
    }

    if (library[index].quantity > 1) {
        library[index].quantity--;
        if (library[index].borrowed > library[index].quantity) {
            library[index].borrowed = library[index].quantity;
        }
        printf("数量减少到%d。 \n", library[index].quantity);
    } else {
        library[index].deleted = 1;
        hashDelete(isbn);
        printf("图书已删除! \n");
    }
    system("pause");
}

/*借阅功能*/

```

```

void borrowBook(void) {
    char isbn[20];
    printf("\n 请输入要借阅的 ISBN: ");
    fgets(isbn, sizeof(isbn), stdin);
    isbn[strcspn(isbn, "\n")] = '\0';

    int index = hashSearch(isbn);
    if (index == -1 || library[index].deleted) {
        printf("未找到该 ISBN 的图书! \n");
        system("pause");
        return;
    }

    if (library[index].borrowed >= library[index].quantity) {
        printf("所有副本都已被借出! \n");
    } else {
        library[index].borrowed++;
        printf("借阅成功! 当前在库%d 本, 已借出%d 本。 \n",
            library[index].quantity - library[index].borrowed,
            library[index].borrowed);
    }
    system("pause");
}

/* 排序功能 */
int compareByTitle(const void *a, const void *b) {
    return strcmp(((struct Book*)a)->title, ((struct Book*)b)->title);
}

int compareByAuthor(const void *a, const void *b) {
    return strcmp(((struct Book*)a)->author, ((struct Book*)b)->author);
}

int compareByISBN(const void *a, const void *b) {
    return strcmp(((struct Book*)a)->isbn, ((struct Book*)b)->isbn);
}

void rebuildHashTable(void) {
    // 清空哈希表
    for (int i = 0; i < hashTable->capacity; i++) {
        struct HashEntry* entry = hashTable->entries[i];
        while (entry != NULL) {
            struct HashEntry* temp = entry;
            entry = entry->next;
        }
    }
}

```

```

        free(temp);
        hashTable->size--;
    }
    hashTable->entries[i] = NULL;
}

// 重新插入有效条目
for (int i = 0; i < bookCount; i++) {
    if (!library[i].deleted) {
        hashInsert(library[i].isbn, i);
    }
}
}

void sortBooks(void) {
    struct Book temp[MAX_BOOKS];
    int count = 0;
    for (int i = 0; i < bookCount; i++) {
        if (!library[i].deleted) {
            temp[count++] = library[i];
        }
    }

    if (count == 0) {
        printf("没有可排序的书籍! \n");
        system("pause");
        return;
    }

    printf("\n 请选择排序方式: \n1.书名 2.作者 3.ISBN\n 选择: ");
    int choice;
    scanf("%d", &choice);
    getchar();

    switch(choice) {
        case 1: qsort(temp, count, sizeof(struct Book), compareByTitle); break;
        case 2: qsort(temp, count, sizeof(struct Book), compareByAuthor); break;
        case 3: qsort(temp, count, sizeof(struct Book), compareByISBN); break;
        default: printf("无效选择! \n"); system("pause"); return;
    }

    // 更新排序结果并重建哈希表
    int j = 0;
    for (int i = 0; i < bookCount; i++) {

```

```

        if (!library[i].deleted) {
            library[i] = temp[j++];
        }
    }
    rebuildHashTable();

    printf("排序完成! \n");
    system("pause");
}

/* 界面系统 */
void Menu(void) {
    system("cls");
    printf("\n===== 图书管理系统 =====\n");
    printf("1. 登记图书\t2. 查看全部书籍\n");
    printf("3. 搜索书籍\t4. 删除书籍\n");
    printf("5. 借阅书籍\t6. 书籍排序\n");
    printf("7. 退出系统\n");
    printf("===== \n");
    printf("请选择操作: ");
}

int main() {
    hashTable = createHashTable(INITIAL_CAPACITY);

    while(1) {
        Menu();
        int choice;
        scanf("%d", &choice);
        getchar();

        switch(choice) {
            case 1: addBook(); break;
            case 2: displayBooks(); break;
            case 3: searchBook_S(); break;
            case 4: deleteBook(); break;
            case 5: borrowBook(); break;
            case 6: sortBooks(); break;
            case 7:
                printf("\n 感谢使用, 再见! \n");
                // 释放哈希表内存
                for (int i = 0; i < hashTable->capacity; i++) {
                    struct HashEntry* entry = hashTable->entries[i];
                    while (entry != NULL) {

```

```

        struct HashEntry* temp = entry;
        entry = entry->next;
        free(temp);
    }
}
free(hashTable->entries);
free(hashTable);
exit(0);
default:
    printf("无效输入，请重新选择! \n");
    system("pause");
}
}
return 0;
}

```

【实验结果】

1. 添加图书操作

(1) 若添加的图书不存在（以 ISBN 是否相同为标准），添加对应书籍

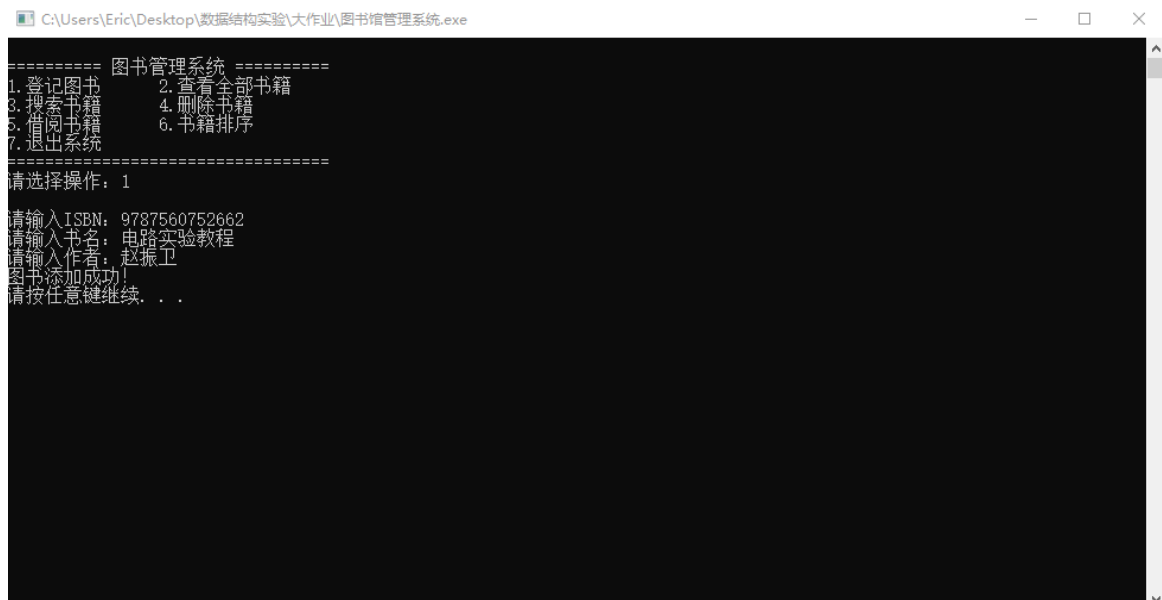


图 1 添加图书

(2) 若添加的书籍已存在，则对应书籍数量加一

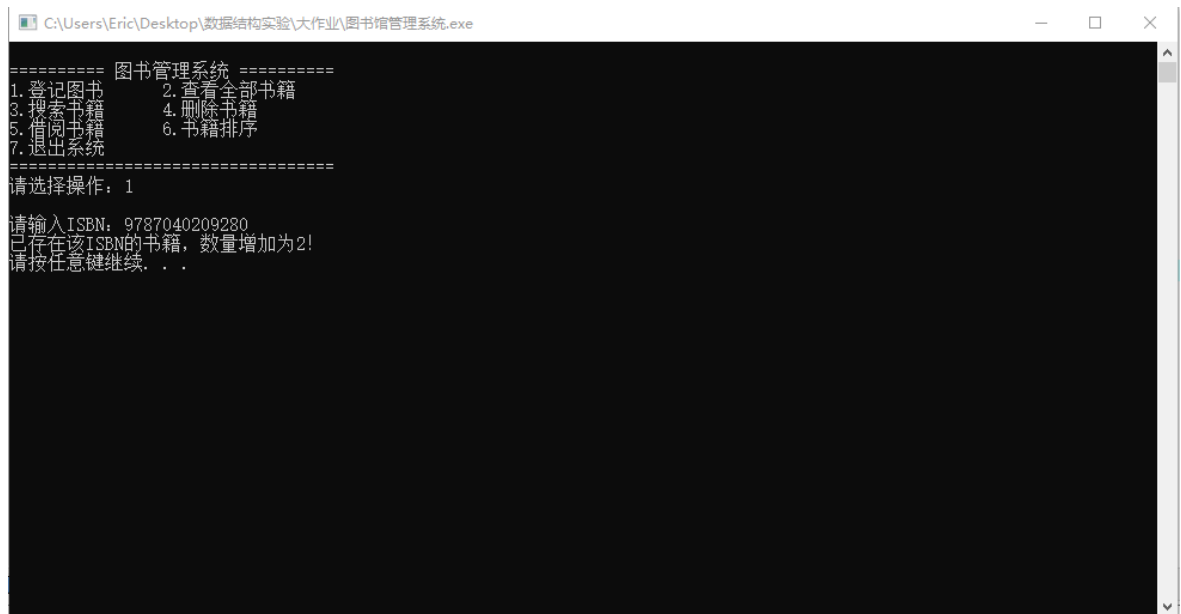


图 2 插入书籍已存在

查看所有图书效果如下:



图 3 全部图书

可以发现，所有图书均正常插入，且添加图书逻辑已经比较完善了。

2. 查看全部图书操作



书名	作者	ISBN	总数	在库	借出
操作系统概念	Abraham Silberschatz	9787040209280	2	2	0
传感器与检测技术 (第五版)	徐科军	9787121406669	1	1	0
电路分析基础	许宏亮	9787302632931	1	1	0
电路实验教程	赵振卫	9787560752662	1	1	0
电子工艺技术基础与实践	韩民	9787560778983	1	1	0
数字物理方法	梁良森	9787040514575	1	1	0
高频电子线路	杨竟清	9787111520573	1	1	0
冯其波	光子测量技术与应用	9787302171362	1	1	0

请按任意键继续...

图 4 查看全部图书功能

可以发现，功能已构建完善

3. 图书删除

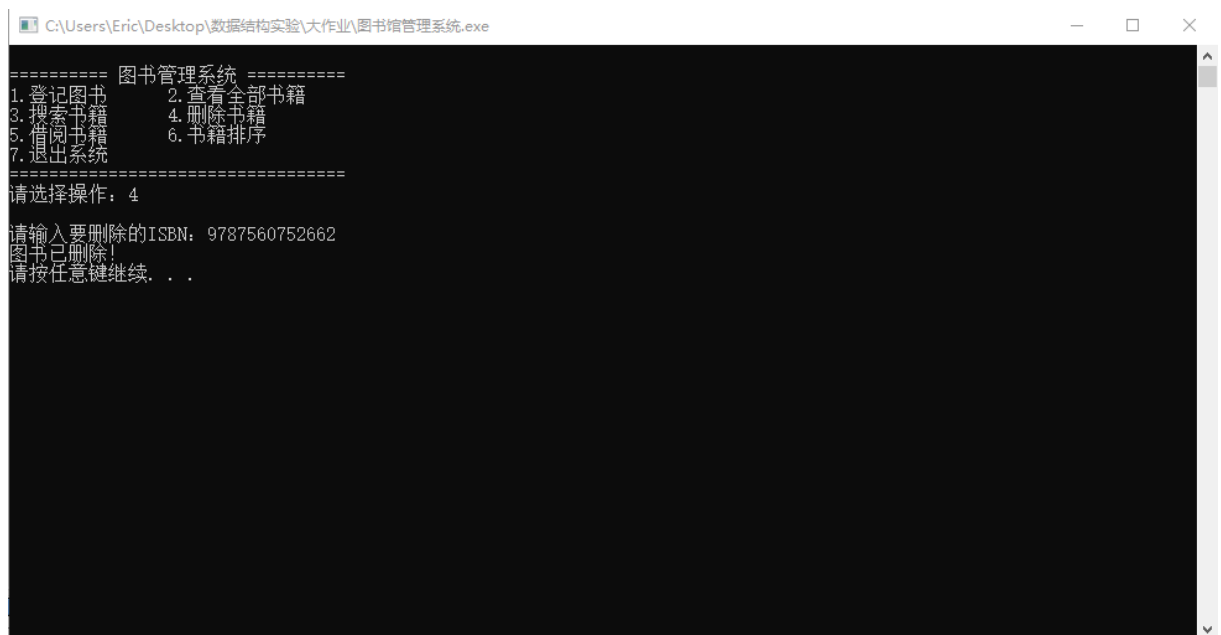


图 5 删除图书操作

在执行查看全部图书操作后，

书名	作者	ISBN	总数	在库	借出
操作系统概念	Abraham Silberschatz	9787040209280	2	2	0
传感器与检测技术（第五版）	徐科军	9787121406669	1	1	0
电路分析基础	许宏吉	9787302632931	1	1	0
电子工艺技术基础与实践	韩民	9787560778983	1	1	0
数字物理方法	梁日森	9787040514575	1	1	0
高频电子线路	杨露清	9787111520573	1	1	0
冯其波	光学测量技术与应用	9787302171362	1	1	0

请按任意键继续...

图 6 图书删除结果

可以发现，图书已经成功删除

4. 图书借阅

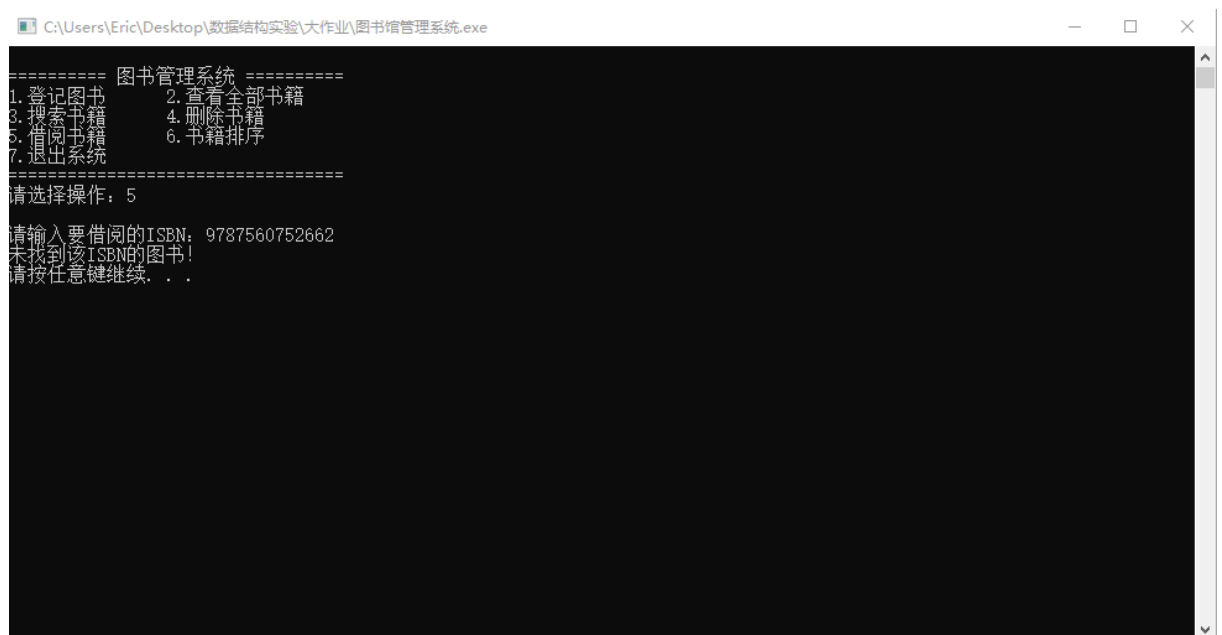


图 7 借阅已删除的书籍图书搜索

可以发现，无法借阅已删除的书籍，逻辑正确

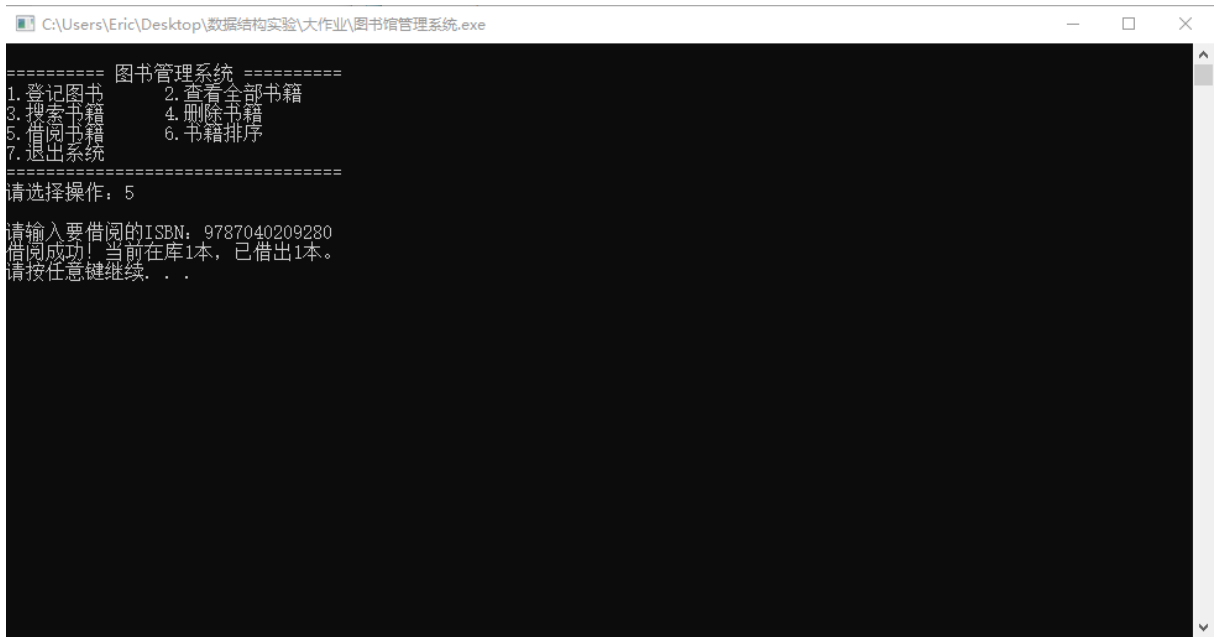


图 8 借阅书籍

可以发现，成功借阅书籍，逻辑正确

若借阅次数超过总馆存：

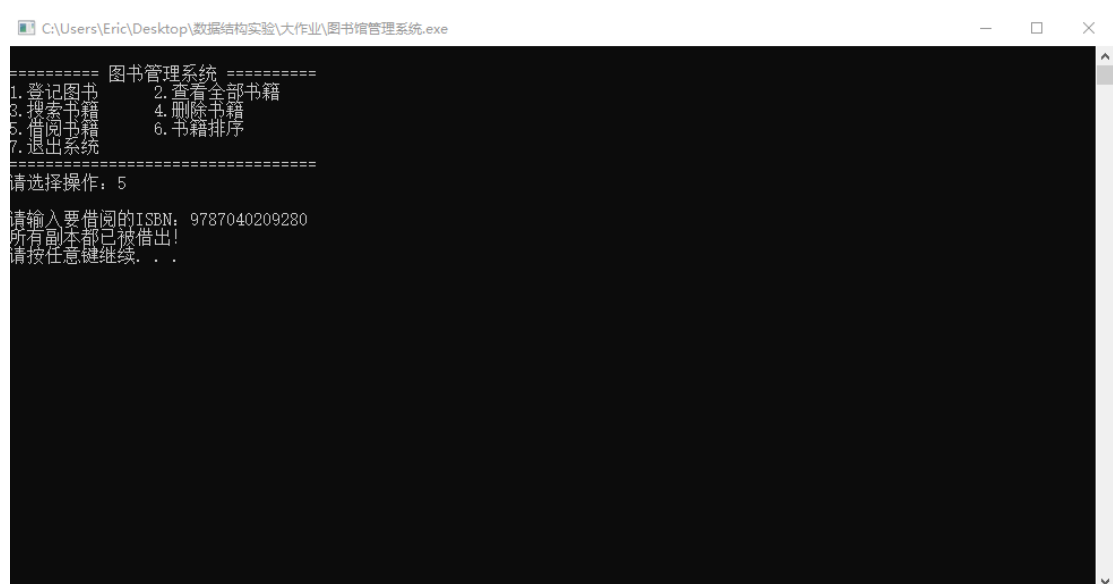


图 9 借阅失败

则借阅失败，逻辑正确。

5. 图书排序

一共设计了 3 种排序方式，书名，人名及 ISBN

(1) 按人名

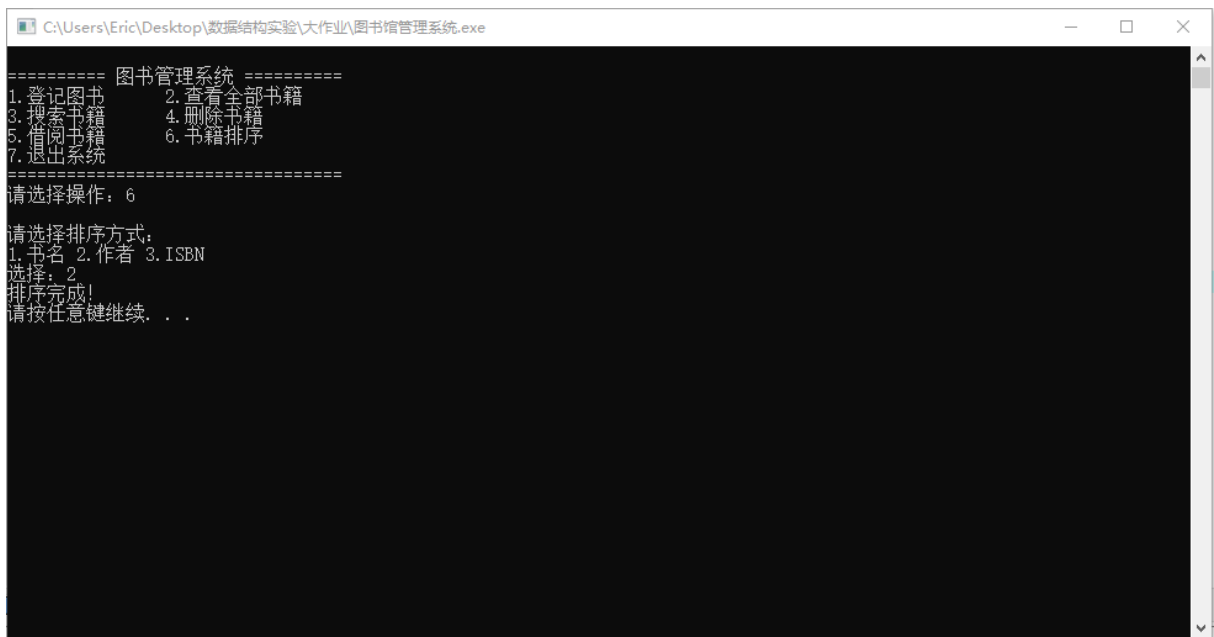


图 10 按人名



图 11 排序结果

成功按人名拼音首字母排序。

(2) 按书名

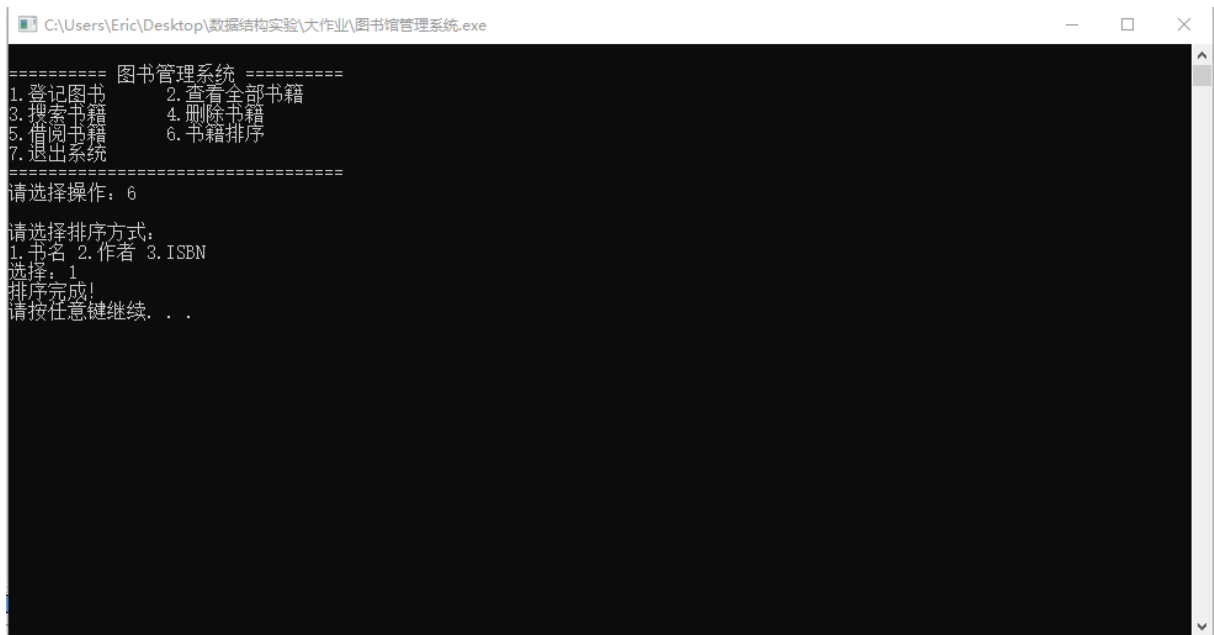


图 12 按书名排序



图 13 排序结果

成功按书名拼音首字母排序。

(3) 按 ISBN

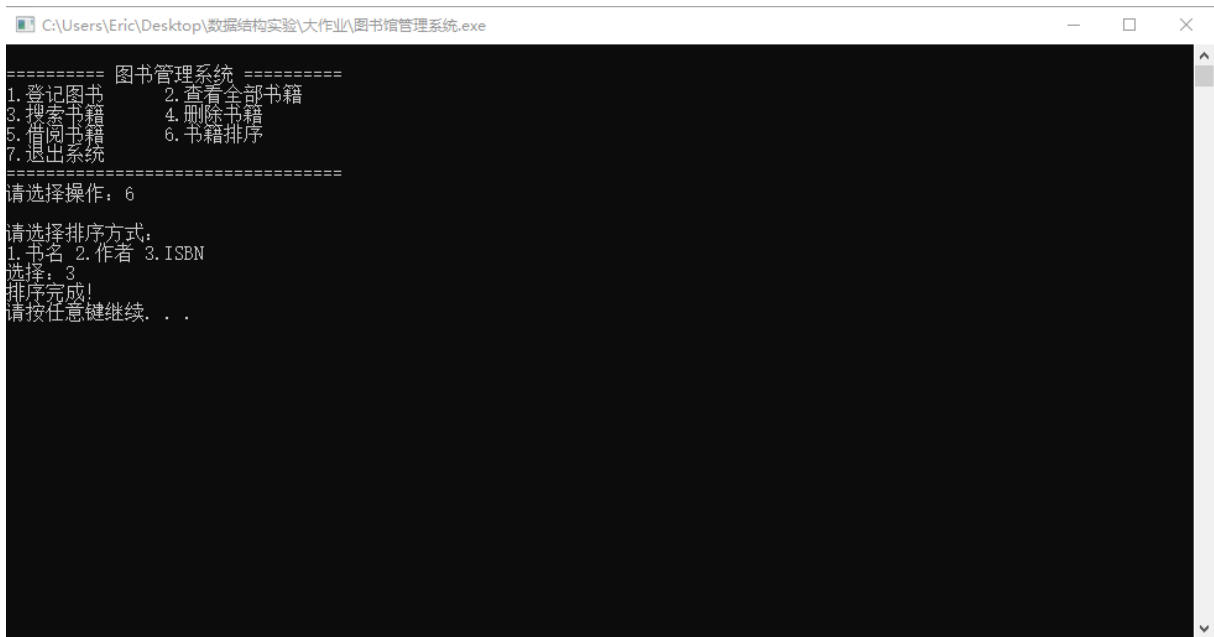


图 14 按 ISBN



图 15 排序结果

成功按 ISBN 排序。

6. 图书搜索

我构造了两种搜索方式:分别是 ISBN 快速搜索和关键词模糊搜索

(1) ISBN 快速搜索

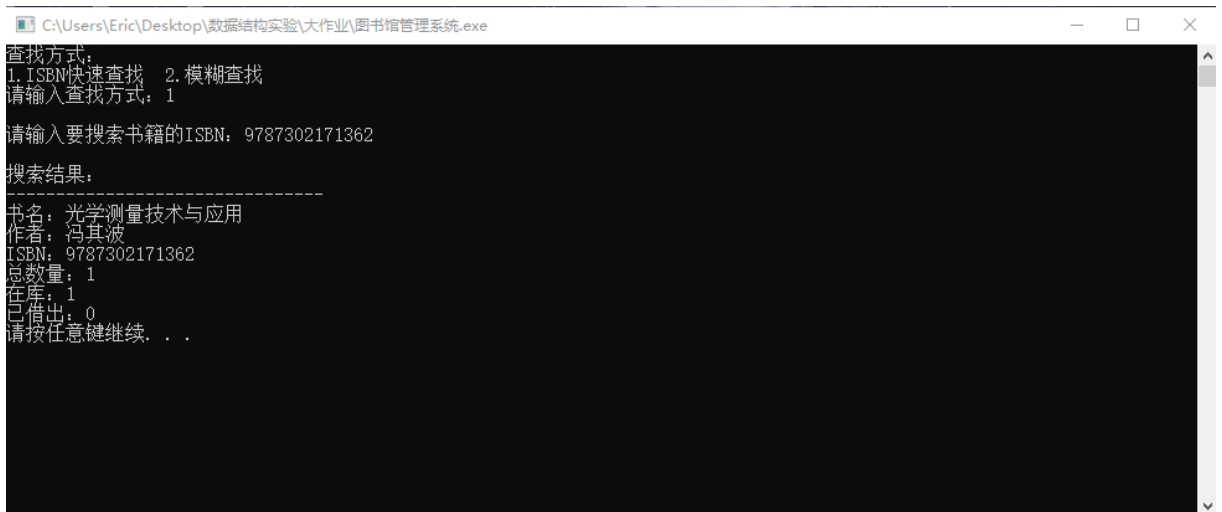
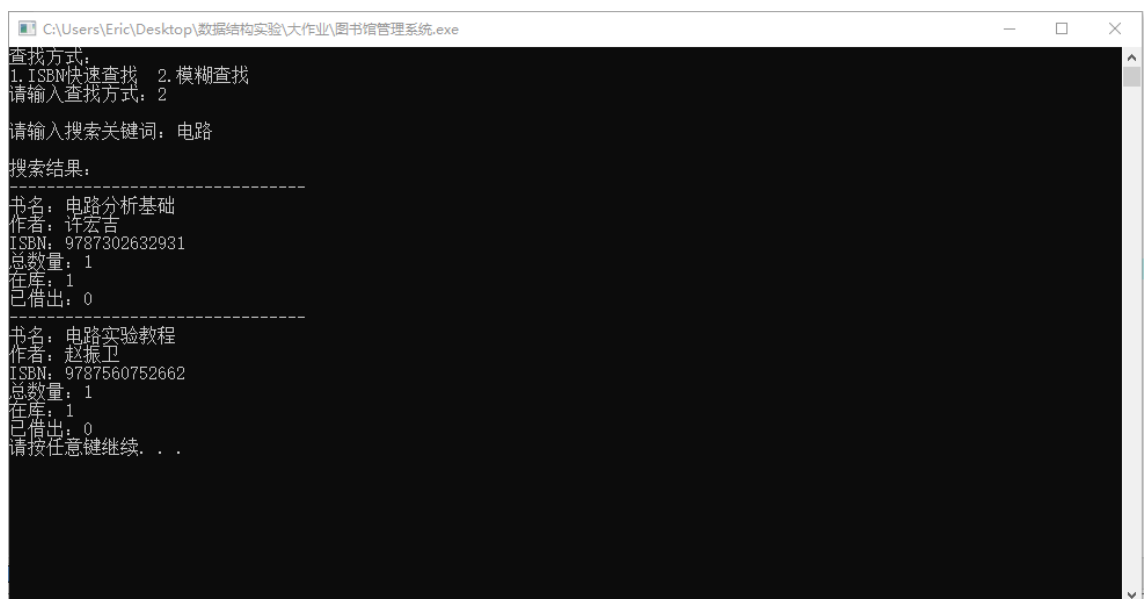


图 16 ISBN 检索结果

成功检索出对应 ISBN 的书籍。

(2) 关键词模糊搜索



成功将所有图书与电路相关的书籍检索了出来

7. 退出系统

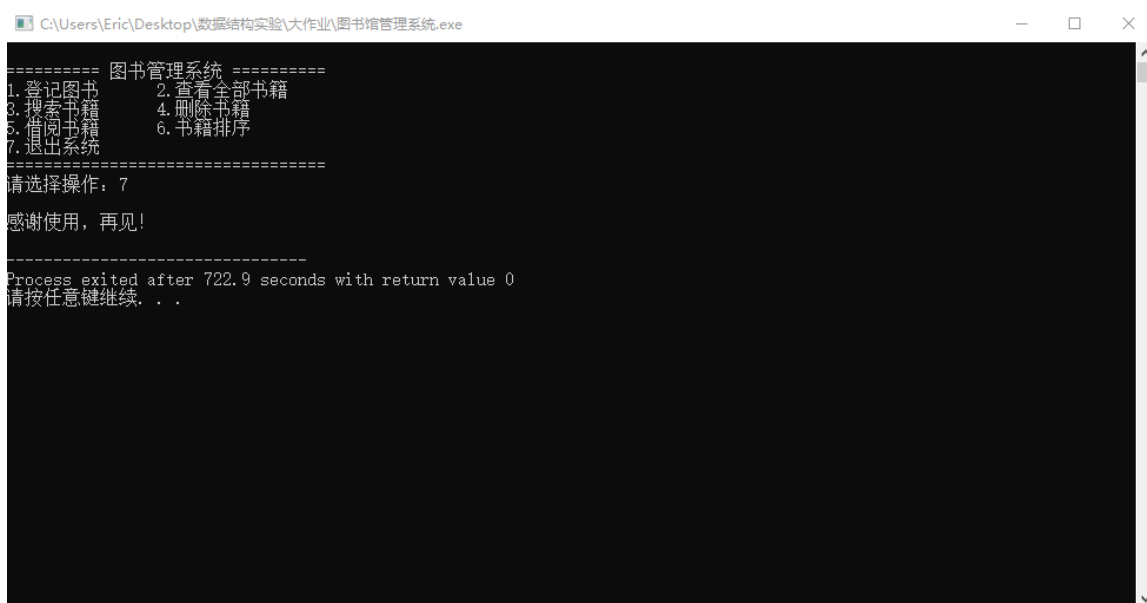


图 17 退出系统

可以发现，退出系统功能已构建完善。

【实验心得】

在不断细化我最初对于该项目的构想，并将其一一实现出来的过程中，我也遇到了许多困难：

1. 删除操作的实现

在实现删除操作的过程中，我发现对于数组来说，删除一个元素并保证空间不被浪费，这要求每删除一个元素，便把其后的元素前移一位，这样做的时间复杂度太高了，为 $O(n)$ ，为了改进这个算法，我们设置了一个状态表（类似哈希表的操作），标记已删除的元素，下次优先插入已删除的元素处。

```
struct Book {  
    char title[100];  
    char author[50];  
    char isbn[20];  
    int borrowed;  
    int deleted;  
    int quantity;  
};
```

2. 哈希表冲突的解决

对于哈希值的构建，我们唯一能够使用的就是 ISBN 了，利用 ISBN 作为哈希值来源，查阅 Java 对应函数库中对于 ISBN 的处理以及相关博客，我们发现大家都在使用 31 这一个质数来将数据尽量分散开，因此，参照其他函数库对 ISBN 的处理方式，我们将 ISBN 的每一位都乘以 31^k 次方（k

为该数在 ISBN 的位次)，最后使用除留余数法，实现哈希。

```
int hash(char* isbn, int capacity) {
    unsigned long hash_val = 0;
    while (*isbn) {
        hash_val = hash_val * 31 + *isbn++;
    }
    return hash_val % capacity;
}
```

在课程实验中，我们使用的是线性探测法来解决冲突问题，但实际上，该算法会导致数据堆积的副作用，即当哈希表比较大时，数据会在零星地方堆积，而链地址法不会，而且其可以负担更大的负载因子，同时其删除操作（对一个桶重新哈希）和扩容操作也简化了，因此在大容量数据下，我们应该使用稳定性更好的链地址法。（实际算法实现并不复杂）。

```
/* 哈希表插入 */
void hashInsert(char* isbn, int index) {
    // 检查负载因子
    if ((float)hashTable->size / hashTable->capacity > LOAD_FACTOR) {
        resize(hashTable);
    }

    int hashVal = hash(isbn, hashTable->capacity);
    struct HashEntry* newEntry = (struct HashEntry*)malloc(sizeof(struct HashEntry));
    strcpy(newEntry->isbn, isbn);
    newEntry->index = index;
    newEntry->next = hashTable->entries[hashVal];
    hashTable->entries[hashVal] = newEntry;
    hashTable->size++;
}
```

3.搜索算法的完成

对于搜索算法，我最先实现的是基于 ISBN 的哈希搜索，这种算法虽然在时间效率非常高，但是，在实际应用中，我们可能只是想要搜索相关的关键词来检索书籍，因此，有必要进行关键词的搜索。

对于关键词搜索，我们必须得依靠 C 语言<string.h>自带的字符串查找函数 strstr,它在找到对应字符串时会返回一个指针，我们只用遍历一遍书库，返回指针非空就可以把对应的书籍打印出来。

```
void searchBook_Blurred(void) {
    char keyword[100];
    printf("\n 请输入搜索关键词: ");
```

```

int c;
while ((c = getchar()) != '\n' && c != EOF); // 清空输入缓冲区
fgets(keyword, sizeof(keyword), stdin);
keyword[strcspn(keyword, "\n")] = '\0';

printf("\n 搜索结果: \n");
int found = 0;
for(int i = 0; i < bookCount; i++) {
    if (library[i].deleted) continue;
    if (strstr(library[i].title, keyword) ||
        strstr(library[i].author, keyword) ||
        strstr(library[i].isbn, keyword)) {
        printf("-----\n");
        printf("书名:%s\n 作者:%s\nISBN:%s\n 总数量:%d\n 在库:%d\n 已借出:%d\n",
            library[i].title,
            library[i].author,
            library[i].isbn,
            library[i].quantity,
            library[i].quantity - library[i].borrowed,
            library[i].borrowed);
        found = 1;
    }
}
if(!found) printf("未找到相关书籍! \n");
system("pause");
}

```

4.排序算法的实现

排序算法我们已经在课程实验中已经完成了，这里仍待我们解决的是如何比较权值的大小，经过搜索，我发现 C 语言自带 strcmp 字符串比较函数，它逐个比较，对于中文也内置了编码库来对于，因此我们使用 strcmp 字符串比较函数比较权值大小，稍微修改快速排序的函数，即可完成任务。

```

int compareByTitle(const void *a, const void *b) {
    return strcmp(((struct Book*)a)->title, ((struct Book*)b)->title);
}

int compareByAuthor(const void *a, const void *b) {
    return strcmp(((struct Book*)a)->author, ((struct Book*)b)->author);
}

int compareByISBN(const void *a, const void *b) {
    return strcmp(((struct Book*)a)->isbn, ((struct Book*)b)->isbn);
}

```

5. 借阅及添加书籍功能的完善

在完成借阅书籍功能时，我发现自己并没有设置书本的数目，可以无穷无尽地借，这与实际应用情况相去甚远，因此，我重新为所有的书添加了数量，在添加书本时，如果遇到 ISBN 重复的书，不会报错，而是将其数量加 1，并且我改进了查看所有书籍功能，可以同时看到现在在馆书籍的总量，已借出的量，以及在库量。

总的来说，本次大作业是我实践课上所学算法的一个结果，现在我写完一句代码更不仅仅关注它能否实现它应有的功能，而是更加关注它的时间效率以及空间效率，还有实际生活中是如何运用的，在本次的图书管理系统中，我认为基于散列查找的该项目，能够做到 $O(1)$ 的高效查找，对频繁操作的动态系统稳定性强，通过哈希表与数组的组合，在空间效率和查询速度之间取得了良好平衡。